Casual Flying High Customer: Jane

First name:	Jane
Last name:	Smith
Email:	jane@acme.com
DOB:	29/08/1981
Phone:	0123456789
Address:	10 Partridge Street, Dandenong



Background: Jane is an account manager for a pharmaceutical company in Melbourne.
Family: Married with 2 children, a 5 year old boy and a 2 year old girl
Preferred communication: email and phone
Seating preference: Aisle

Flying profile: Jane flies 2-3 times per month for her job. She takes whatever airline proposes the most attractive deal.

Figure 7.7 A persona is a fictional character meant to represent a category of user of the system.

In the implementation of the first step, you can use the name *Jane* ❶ as an indicator to inject all of the relevant data about this persona into the test database. In subsequent steps, you can use the persona name to identify the data set you need to use for the rest of the scenario.

When you initially configure this test data, you might do something like this:

```
Persona persona;

@Given("$name is not a Frequent Flyer member")
public void givenANonFrequentFlyerMember(String name) {
    persona = Persona.withName(name);
    persona.setStatus(NonMember);
    persona.save();
}
```

A special test-specific domain object to store persona data. ❶

Fetch this persona data by name. ❷

Strip this persona of their membership. ❸

Save the updated persona. ❹

The `Persona` class ❶ is created for these tests to define the known personas and inject them into the test database. In the method body, you retrieve the corresponding persona data ❷, ensure that this persona isn't a Frequent Flyer member ❸, and then save the updated persona to the test database ❹. Note that not all persona objects would need to go into the test database—some might simply be configured in the initial steps and then used in subsequent steps. In either case, the primary goal is to make the scenario more readable and easier to maintain by removing any unnecessary clutter.

You can also use the persona data as a kind of template, changing other fields as required. For example, suppose you needed to implement the following step:

```
Given Jane is 60 years old
```